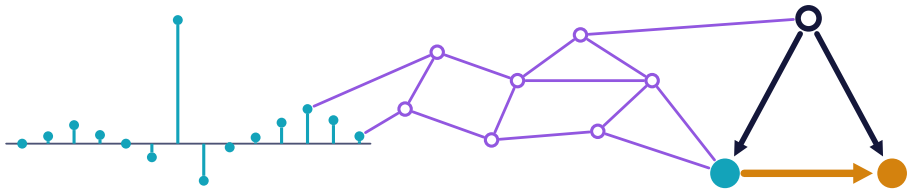
Redes recurrentes y LSTM

Memoria, retropropagación en el tiempo y compuertas

P. E. Caicedo-Rodríguez
Centro de Estudios en Biomédica y Biotecnología
24 de octubre de 2026



Por qué secuencias



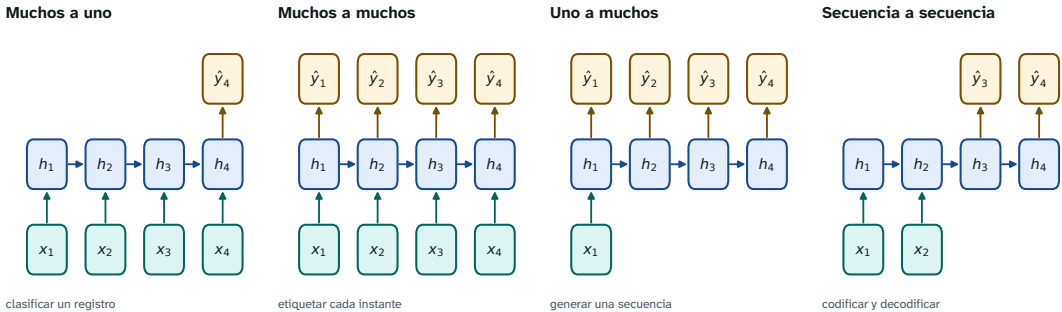
Datos con orden

En una señal biomédica el orden importa: el mismo conjunto de muestras, desordenado, ya no es la misma señal.

- EMG, ECG, EEG: el ritmo y la forma de los potenciales están en cómo cambia la señal en el tiempo
- Una red densa necesita una entrada de **longitud fija** y no comparte nada entre instantes
- Una red convolucional comparte pesos, pero ve una **ventana local** fija

Queremos un modelo que lea la secuencia paso a paso, con los mismos pesos en cada paso, y que lleve un resumen de lo que ya vio.

Cuatro formas de usar una secuencia



Forma	Ejemplo biomédico
Muchos a uno	clasificar un registro de EMG como sano, miopatía o neuropatía
Muchos a muchos	marcar cada latido de un ECG como normal o ectópico

La red recurrente simple



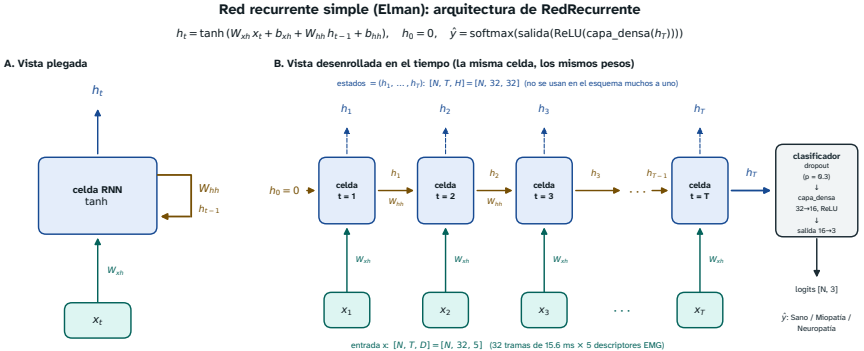
El estado oculto es la memoria

En cada paso la red combina la entrada actual con su estado anterior:

$$h_t = \tanh\left(W_{xh}\, x_t + b_{xh} + W_{hh}\, h_{t-1} + b_{hh}\right), \qquad h_0 = 0$$

Símbolo	Forma	Papel
x_t	D	entrada en el paso t
h_t	H	estado oculto: resumen de $x_1, \dots, x_t$
W_{xh}	$H \times D$	cómo entra el presente
W_{hh}	$H \times H$	cómo pasa el pasado: la recurrencia

Desenrollar en el tiempo



Un ejemplo a mano

Una red con $D = H = 1$, $w_{xh} = 0.5$, $w_{hh} = 0.8$, sin sesgos, y la entrada $x = (1, 0, 0, 0)$:

t	**Cálculo**	h_t
1	$\tanh(0.5 \cdot 1 + 0.8 \cdot 0)$	**0.462**
2	$\tanh(0.8 \cdot 0.462)$	**0.354**
3	$\tanh(0.8 \cdot 0.354)$	**0.276**
4	$\tanh(0.8 \cdot 0.276)$	**0.217**

El pulso del primer paso se recuerda, pero se apaga: cada paso lo multiplica por algo menor que 1.

Salida, pérdida y parámetros

Salida (muchos a uno): $\hat{y} = \text{softmax}(W_{hy}\, h_T + b_y)$ y pérdida de entropía cruzada $\mathcal{L} = -\log \hat{y}_c$.

Muchos a muchos: una salida por paso y $\mathcal{L} = \sum_t \ell_t$.

Parámetros de la celda, independientes de la longitud T:

$$H\,(D + H + 2) \quad \Rightarrow \quad D = 5,\ H = 32 : \ 32 \cdot 39 = 1\,248$$

Entrenar: retropropagación en el tiempo



La idea de BPTT

Se desenrolla la red, se calcula la pérdida y se retropropaga a través de todos los pasos. Como los pesos son compartidos, su gradiente es la **suma** de las contribuciones de cada paso:

$$\frac{\partial \mathcal{L}}{\partial W_{hh}} = \sum_{t=1}^{T} \delta_t \odot (1 - h_t^2)\, h_{t-1}^{\top}, \qquad \delta_t = \frac{\partial \mathcal{L}}{\partial h_t}$$

El error llega a h_t por dos caminos: su propia salida y el paso siguiente.

$$\delta_t = \frac{\partial \ell_t}{\partial h_t} + W_{hh}^{\top}\, \mathbf{diag}(1 - h_{t+1}^2)\, \delta_{t+1}$$

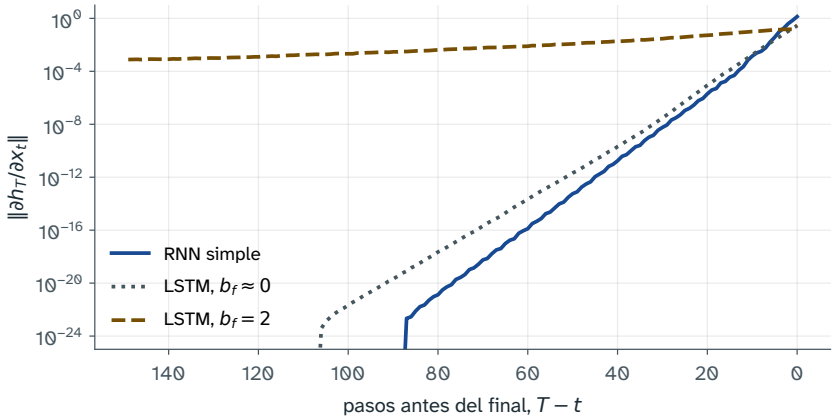
Un producto de jacobianos

Para saber cuánto influye un estado lejano en el final:

$$\frac{\partial h_T}{\partial h_k} = \prod_{t=k+1}^{T} \text{diag}(1 - h_t^2)\, W_{hh} \qquad \Rightarrow \qquad \left\| \frac{\partial h_T}{\partial h_k} \right\| \leq \|W_{hh}\|^{T-k}$$

- Si $\|W_{hh}\| < 1$: el producto se desvanece exponencialmente
- Si el mayor valor singular de W_{hh} pasa de 1: el producto puede explotar
- La saturación de tanh ($1 - h^2 \approx 0$) empuja hacia el desvanecimiento

El gradiente se desvanece



Redes sin entrenar, secuencias aleatorias de 150 pasos, promedio de 20 secuencias.

Red 10 pasos atrás 100 pasos atrás

Remedios

Problema	Remedio
Explosión	**recorte de la norma**: si $\|g\| > \tau$, $g \leftarrow \tau\, g/\|g\|$
Secuencias muy largas	**BPTT truncada**: retropropagar solo los últimos k pasos
Desvanecimiento	inicialización ortogonal de W_{hh} y, sobre todo, **cambiar la arquitectura**

```
perdida.backward()
nn.utils.clip_grad_norm_(modelo.parameters(), max_norm=1.0)
optimizador.step()
```

LSTM: memoria con compuertas

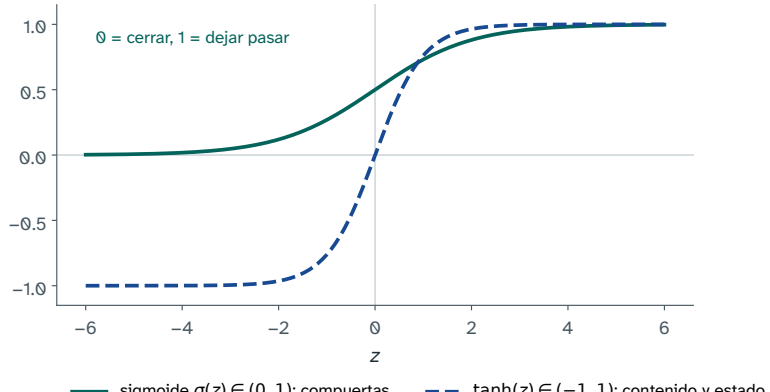


La idea

La LSTM añade una **memoria** c_t que se actualiza sumando, no multiplicando, y tres **compuertas** que deciden qué pasa por ella:

- Olvido f_t: cuánto conservar de c_{t-1}
- Entrada i_t: cuánto escribir del contenido nuevo
- Candidato g_t: qué contenido nuevo se propone
- Salida o_t: cuánto de la memoria se muestra en h_t

Una compuerta es una sigmoide que multiplica



Una compuerta es un vector de valores en $(0, 1)$ que multiplica elemento a

Las ecuaciones

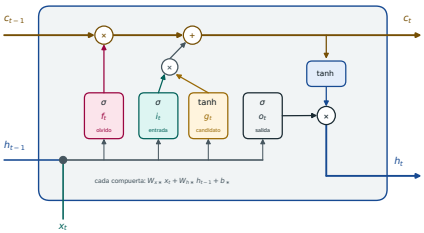
$$
\begin{aligned}
i_t &= \sigma\left(W_{xi}\, x_t + W_{hi}\, h_{t-1} + b_i\right) \\
f_t &= \sigma\left(W_{xf}\, x_t + W_{hf}\, h_{t-1} + b_f\right) \\
g_t &= \tanh\left(W_{xg}\, x_t + W_{hg}\, h_{t-1} + b_g\right) \\
o_t &= \sigma\left(W_{xo}\, x_t + W_{ho}\, h_{t-1} + b_o\right) \\
c_t &= f_t \odot c_{t-1} + i_t \odot g_t \\
h_t &= o_t \odot \tanh(c_t)
\end{aligned}
$$

La celda por dentro

Red LSTM: la celda por dentro y la arquitectura de RedLSTM

La celda guarda dos estados: la memoria c_t (línea superior, camino aditivo) y el estado oculto h_t (lo que ve la capa siguiente)

A. Una celda LSTM en el paso t



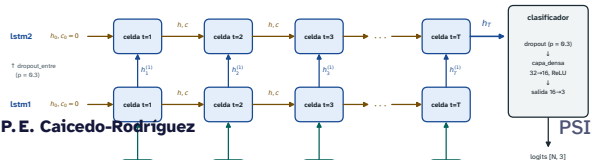
Ecuaciones (secciones 3.4 y 3.7)

$f_t = \sigma(W_{xf}x_t + W_{hf}h_{t-1} + b_f)$	compuerta de olvido: cuánto conservar
$i_t = \sigma(W_{xi}x_t + W_{hi}h_{t-1} + b_i)$	compuerta de entrada: cuánto escribir
$g_t = \tanh(W_{xg}x_t + W_{hg}h_{t-1} + b_g)$	contenido candidato
$o_t = \sigma(W_{xo}x_t + W_{ho}h_{t-1} + b_o)$	compuerta de salida: cuánto mostrar
$c_t = f_t \odot c_{t-1} + i_t \odot g_t$	actualización aditiva de la memoria
$h_t = o_t \odot \tanh(c_t)$	estado oculto

Gradiente por la memoria: $\partial c_t/\partial c_{t-1} = \mathrm{diag}(f_t)$; con $f_t = 1$ no se desvanece.

σ = sigmoide (valores en (0, 1)). $\odot$ = producto elemento a elemento. $\times$ y $+$ en los nodos idem.

B. RedLSTM desenrollada: dos capas apiladas (lstm1 → lstm2) y clasificador



C. Parámetros (H = 32)

tensor	forma	parám.
`lstm1.weight_ih_l0`	[128, 5]	640
`lstm1.weight_hh_l0`	[128, 32]	4 096
`lstm1.bias_ih_l0 / hh`	2 × [128]	256
`lstm2.weight_ih_l0`	[128, 32]	4 096
`lstm2.weight_hh_l0`	[128, 32]	4 096
`lstm2.bias_ih_l0 / hh`	2 × [128]	256

lstm1: $4H(D + H + 2) = 4 \cdot 32 \cdot 39 = 4\,992$

lstm2: $4 \cdot 32 \cdot 64 = 8\,448$

$4H = 128$ filas = bloques apilados en el orden de PyTorch: input, forget, cell, output.

Por qué no se desvanece

Por el camino de la memoria la derivada es, sin contar los términos indirectos por h_{t-1}:

$$\frac{\partial c_t}{\partial c_{t-1}} = \text{diag}(f_t) \qquad \Rightarrow \qquad \frac{\partial c_T}{\partial c_k} \approx \prod_{t=k+1}^{T} \text{diag}(f_t)$$

Compuerta de olvido	Lo que queda tras 10 pasos
$f = 0.99$	0.90
$f = 0.9$	0.35
RNN con $\|W_{hh}\| = 0.8$ (cota)	0.11

La red aprende cuánto recordar: f_t depende de la entrada y del estado.

Parámetros y convención de PyTorch

$$4H\,(D + H + 2) \quad \Rightarrow \quad D = 5,\ H = 32 :\ 4 \cdot 32 \cdot 39 = 4\,992$$

PyTorch apila las cuatro compuertas en un solo tensor, en el orden i, f, g, o:

Tensor	Forma
`weight_ih_l0`	$[4H, D]$
`weight_hh_l0`	$[4H, H]$
`bias_ih_l0`, `bias_hh_l0`	$[4H]$ cada uno

```
with torch.no_grad():
    lstm.bias_hh_l0[H:2 * H] = 1.0   # bloque f: empezar recordando
```

Un paso de la LSTM, escrito a mano

```python
def paso_lstm(x_t, h, c, W_x, W_h, b):
    z = x_t @ W_x.T + h @ W_h.T + b      # [N, 4H]
    i, f, g, o = z.chunk(4, dim=1)       # orden de PyTorch
    i = torch.sigmoid(i)
    f = torch.sigmoid(f)
    g = torch.tanh(g)
    o = torch.sigmoid(o)
    c = f * c + i * g                    # memoria
    h = o * torch.tanh(c)                # estado oculto
    return h, c
```

Con los pesos de `nn.LSTM`, este paso reproduce su salida: es un buen ejercicio de comprobación.

Variantes y práctica



GRU: dos compuertas, sin memoria aparte

$$
\begin{aligned}
r_t &= \sigma\left(W_{xr}\, x_t + W_{hr}\, h_{t-1} + b_r\right) \\
z_t &= \sigma\left(W_{xz}\, x_t + W_{hz}\, h_{t-1} + b_z\right) \\
n_t &= \tanh\left(W_{xn}\, x_t + b_{xn} + r_t \odot \left(W_{hn}\, h_{t-1} + b_{hn}\right)\right) \\
h_t &= (1 - z_t) \odot n_t + z_t \odot h_{t-1}
\end{aligned}
$$

- r_t (reinicio) decide cuánto pasado usar para el candidato
- z_t (actualización) mezcla el estado anterior con el nuevo
- $3H\,(D + H + 2)$ parámetros: 3 744 con $D = 5$, $H = 32$

Apilar y leer en los dos sentidos

- **Apilada**: la secuencia completa de estados de la capa 1, $(h_1^{(1)}, \dots, h_T^{(1)})$, es la entrada de la capa 2
- **Bidireccional**: una red lee de 1 a T y otra de T a 1; se concatenan los estados: $[\vec{h}_t; \overleftarrow{h}_t]$, de tamaño $2H$
- Cuidado: la bidireccional usa el futuro; sirve para analizar un registro completo, no para decidir en tiempo real

En PyTorch: formas

```
lstm = nn.LSTM(input_size=5, hidden_size=32, batch_first=True)
x = torch.randn(8, 32, 5)                # [N, T, D]
salidas, (h_n, c_n) = lstm(x)
salidas.shape                            # [8, 32, 32]  todos los h_t
h_n.shape                                # [1, 8, 32]   h_T por capa
```

Esquema	Qué se usa
Muchos a uno	`h_n[-1]` o `salidas[:, -1]`
Muchos a muchos	`salidas` completa

Un modelo completo, capa por capa

```python
class RedLSTM(nn.Module):
    def __init__(self, n_entradas=5, n_ocultas=32, n_clases=3):
        super().__init__()
        self.lstm1 = nn.LSTM(n_entradas, n_ocultas, batch_first=True)
        self.lstm2 = nn.LSTM(n_ocultas, n_ocultas, batch_first=True)
        self.dropout = nn.Dropout(0.3)
        self.capa_densa = nn.Linear(n_ocultas, 16)
        self.salida = nn.Linear(16, n_clases)

    def forward(self, x):                    # x: [N, T, D]
        h1, _ = self.lstm1(x)                # [N, T, H]
        h1 = self.dropout(h1)
        _, (h_T, _) = self.lstm2(h1)         # h_T: [1, N, H]
        z = self.dropout(h_T[-1])            # [N, H]
        z = torch.relu(self.capa_densa(z))   # [N, 16]
```

Antes de creer en el resultado

- Normalizar las entradas con estadísticas del entrenamiento
- Dividir por paciente, no por ventana
- Recortar el gradiente e iniciar b_f en 1
- Comparar con una línea base sin tiempo: media y desviación de cada descriptor más una regresión logística
- **Desordenar los pasos de tiempo** en la prueba: si el acierto no cae, el modelo no está usando el orden

Cierre



Resumen

	RNN simple	LSTM	GRU
Estado	h_t	h_t y memoria c_t	h_t
Compuertas	ninguna	olvido, entrada, salida	reinicio, actualización
Parámetros ($D = 5$, $H = 32$)	1 248	4 992	3 744
Memoria larga	se desvanece	la regula f_t	la regula z_t
Cuándo usarla	secuencias cortas, línea base	opción por defecto	menos datos o menos cómputo

Para esta semana

Laboratorio

Escribir `paso_lstm` a mano, comprobar que reproduce `nn.LSTM` con los mismos pesos y medir $\|\partial h_T / \partial x_t\|$ con $b_f = 0$, 1 y 2.

1. Rehacer a mano el ejemplo de $w_{hh} = 0.8$ con $w_{hh} = 1.5$. ¿Qué pasa con $1 - h_t^2$?
2. Deducir $4H(D + H + 2)$ y $3H(D + H + 2)$ a partir de las ecuaciones.
3. Mostrar que $\partial c_t / \partial c_{t-1} = \text{diag}(f_t)$ si se ignoran los caminos por h_{t-1}.
4. Cambiar la LSTM del ejemplo de EMG por una GRU y comparar parámetros y acierto.